Testing PHOTON locally on an Android phone

PHOTON | Computer Networks (CSC-303) | Group 13

How to run the backend on your computer, put the app on a phone over USB, and check that photos are shared, including when the app is in the background. Run every command from the project folder (`~/dev/Photon`). You'll use three terminals.

One-time setup on the phone

1. Turn on USB debugging: Settings → About phone → tap **Build number** 7 times, then Settings → System → Developer options → **USB debugging**.
2. Plug the phone in, and on the "Allow USB debugging?" prompt tick **Always allow from this computer**, then tap **Allow**.

Terminal 1: the backend (leave it running)

```
cd ~/dev/Photon
scripts/phone-backend.sh
```

It's ready when it prints `Uvicorn running on http://127.0.0.1:8080`. Ctrl-C stops it.

- The first run sets up the Python environment and builds Redis, which takes a few minutes.
- If it says a port is already in use, another backend is running. Stop it with `pkill -INT -f "bin/photon-dev"`.
- To try GitHub or Google sign-in, pass the credentials (setup in `prehost.md`, section 5):

  ```
  PHOTON_GITHUB_CLIENT_ID=... PHOTON_GITHUB_CLIENT_SECRET=... scripts/phone-backend.sh
  PHOTON_GOOGLE_CLIENT_ID=<web client id> scripts/phone-backend.sh
  ```

  For local use, the GitHub OAuth App's callback URL is `http://127.0.0.1:8080/v1/auth/oauth/return`.

Terminal 2: install the app on the phone

```
cd ~/dev/Photon
scripts/phone-install.sh
```

It forwards ports 8080 and 8081 to the phone over USB (`adb reverse`), then builds, installs and opens PHOTON.

Command	When
`scripts/phone-install.sh`	First time, and after every code change
`scripts/phone-install.sh --reverse`	After replugging the cable or restarting the computer: restores the port forwarding, with no rebuild
`scripts/phone-install.sh --logs`	The same as the first, then follows the app's log
`scripts/phone-install.sh --all`	Several phones connected: set up all of them (see Two phones)
`ANDROID_SERIAL=<serial> scripts/phone-install.sh`	Several phones connected: set up just one (serials from `adb devices`)
`scripts/phone-install.sh --lan <address>`	Phones on Wi-Fi instead of USB (see Over Wi-Fi)

On the phone

1. Sign in: **Developer sign-in** with any name, or GitHub or Google if the backend has them configured.
2. Create a session and note the 8-character code.
3. Allow photos with **Allow all** (not "Select photos"), and allow notifications.
4. If a card asks to let PHOTON run in the background, tap it and allow.
5. A notification "PHOTON · sharing" stays visible while you're in a session. That's the sync service that watches for new photos.

Terminal 3: a second member

The computer acts as another phone:

```
cd ~/dev/Photon
.venv/bin/photon-cli --profile bob login
.venv/bin/photon-cli --profile bob join <CODE>
.venv/bin/photon-cli --profile bob listen --save-dir ./received
```

Leave `listen` running: every photo the phone shares is printed and saved to `./received/`. A second phone works too; install the app on it and join with the code.

What to test

Test	How	Expected
App open	Take a photo while PHOTON is on screen	A line in terminal 3 and a file in `./received/` within a few seconds
Background	Press Home, open the camera, take a photo	The same, within a few seconds
App swiped away	Swipe PHOTON from recents, then take a photo	The same; the sharing notification stays or comes back
Phone receives	In a fourth terminal: `.venv/bin/photon-cli --profile bob upload some.jpg`	It appears on the phone, and in its gallery under **Pictures/PHOTON**
Recall	Recall a shared photo in the app	Terminal 3 prints `PHOTO_RECALLED`
Leave	Tap Leave in the app	Terminal 3 prints the member leaving; sharing stops

Other `photon-cli` commands: `info`, `snapshot`, `catchup`, `recall <sha>`, `download <sha>`, `leave`, `end`.

Two (or more) phones

Both on USB (simplest)

Plug both phones into this computer, accept the USB debugging prompt on each, and keep the backend from terminal 1 running. Then:

```
scripts/phone-install.sh --all            # forward the ports, build once, install and open
    ↪ the app on every phone
scripts/phone-install.sh --reverse --all  # after replugging a cable
```

1. **Phone A:** sign in (Developer sign-in, for example as `alice`) and create a session. The screen shows the code and a QR code.
2. **Phone B:** sign in with a **different** name (for example `bob`) and join with the code, or scan the QR code with the camera.
3. On both phones, allow photos with **Allow all**, and notifications.

What to test:

Test	Expected
Take a photo on A (A open or in the background)	B shows the thumbnail within a few seconds, then saves the original under **Pictures/PHOTON**
Take a photo on B	The same on A
Add the computer as a third member (terminal 3 above)	`listen` receives every photo both phones take
Recall a photo on A	It disappears from B's PHOTON screen (the copy already saved to B's gallery stays; that's by design)
A leaves or ends the session	B sees the member leave, or the session end

Diagnosing one phone of several: `ANDROID_SERIAL=<serial> scripts/phone-diagnose.sh` (serials from `adb devices`). The app log of each: `adb -s <serial> logcat -s PHOTON`.

Over Wi-Fi (a phone that isn't plugged in)

Both phones and this computer must be on the same Wi-Fi. This exposes the development backend, which has no real sign-in, to your network, so only do it on a network you trust.

1. Find this computer's Wi-Fi address: `ip route get 1.1.1.1` shows it after `src` (for example `10.81.65.165`). Ignore the `172.x` addresses from `hostname -I`; they are Docker's internal networks.
2. **Terminal 1:** `scripts/phone-backend.sh --host 0.0.0.0 --public-host 10.81.65.165`
3. On each phone, open `http://10.81.65.165:8080/healthz` in the browser. It must show `ok`. If it doesn't, open the ports (`sudo ufw allow 8080,8081/tcp`). If it still fails, the Wi-Fi keeps devices apart (common on campus and office networks); use USB instead.
4. **Terminal 2:** build the app for that address:

   ```
   scripts/phone-install.sh --lan 10.81.65.165        # add --all if several phones are
       ↪ plugged in
   ```

   It builds the app **for that address**, installs it on any phone that's plugged in, and prints where the APK is.

5. Install it on every phone that step 4 didn't already:
   - **Plugged in by USB** (for example, plugged in after step 4):

     ```
     adb install -r android/app/build/outputs/apk/debug/app-debug.apk
     ```

   - **Not plugged in:** copy `android/app/build/outputs/apk/debug/app-debug.apk` to the phone (cable, Drive, a chat app, anything) and open it, allowing "Install unknown apps" when asked. It updates PHOTON in place and keeps the sign-in.
6. Sign in with a different name on each phone, create a session on one, and join it from the others.

Every phone in the session needs this Wi-Fi build. An app built by the plain `scripts/phone-install.sh` talks to `127.0.0.1`, which over Wi-Fi is the phone itself: the browser check passes, but the app says it can't reach the server. Developer sign-in works the same; GitHub and Google sign-in need their callback set up for this address, so stick to Developer sign-in here. If the computer's address changes (another network, or a reconnect), restart the backend and repeat steps 4 and 5 with the new address.

If something doesn't arrive

```
scripts/phone-diagnose.sh           # read-only report: service, battery, jobs, why the app
    ↪ died, app log
scripts/phone-diagnose.sh --watch   # then follows the app's log live while you take a photo
```

The usual causes, in order:

1. **The backend in terminal 1 has stopped.** Start it again.
2. **The port forwarding is gone** after the cable was replugged or adb restarted. The app's log shows `Failed to connect to /127.0.0.1:8080`. Run `scripts/phone-install.sh --reverse`.
3. **The phone shows "unauthorized"** in the report. Unlock it and accept the USB debugging prompt.
4. **"Sync service: NOT RUNNING"** while you're in a session. Open the app once; it restarts the service. If it keeps stopping, the report's "Why the process last died" and app log sections show why.
5. **Wi-Fi: the browser shows `ok` at `/healthz`, but the app says it can't reach the server.** The app on the phone was built for USB (`127.0.0.1`, which over Wi-Fi is the phone itself). Run `scripts/phone-install.sh --lan <address>` and install the new APK (Over Wi-Fi, steps 4 and 5).

Starting over

- Reset all local data (accounts, sessions, photos, keys): stop the backend and delete `.devdata/`.
- Remove the app from the phone: `adb uninstall com.photon.app`.